ĐỀ CƯƠNG TRIỂN KHAI

Lộ trình xây dựng và triển khai trợ lý nói Việt - Anh

Từ lựa chọn mô hình, chuẩn bị dữ liệu và self-host Qwen đến fine-tune, tích hợp Next.js và vận hành production.

Đối tượng: Nhóm kỹ thuật và quản lý sản phẩm | **Phạm vi:** Pilot đến production

Phiên bản: 1.0 | Cập nhật: 20/08/2026

> **Mục tiêu tổng thể.** Xây dựng một nền tảng AI Việt - Anh có thể tự vận hành: dữ liệu được kiểm soát, mô hình được đo lường trước và sau fine-tune, backend có khả năng chuyển đổi nhà cung cấp, và hệ thống production có cơ chế giám sát, dự phòng và kiểm soát chi phí.

Tổng quan lộ trình

Giai đoạn	Trọng tâm	Đầu ra chính
1	Đánh giá model, VRAM và tổng chi phí sở hữu	Danh sách model phù hợp và phương án hạ tầng
2	Xây nền tảng kỹ năng vận hành	Bộ bài thực hành Linux, Docker và GPU container
3	Xây kho dữ liệu Việt - Anh có kiểm duyệt	Dataset phiên bản hóa, hướng dẫn gán nhãn và bộ đánh giá chuẩn
4	Self-host Qwen và benchmark trước khi huấn luyện	Model server qua API cùng báo cáo baseline
5	Fine-tune bằng SFT kết hợp QLoRA	Adapter, cấu hình huấn luyện và báo cáo đánh giá
6	Tích hợp Next.js và triển khai production	Provider, fallback, monitoring và quy trình phát hành

Giai đoạn 1: Tìm hiểu mô hình, tham số và chi phí hạ tầng

Mục tiêu: Chọn được mô hình và cấu hình phục vụ phù hợp với chất lượng kỳ vọng, tải thực tế và ngân sách của dự án.

Kiến thức về mô hình

- **Kiến trúc và quy mô:** hiểu số lượng tham số, độ dài ngữ cảnh, cơ chế attention, khả năng hỗ trợ tiếng Việt - tiếng Anh và giới hạn giấy phép sử dụng.
- **Độ chính xác số:** so sánh BF16/FP16, INT8 và INT4; đánh giá ảnh hưởng của lượng tử hóa đến VRAM, tốc độ và chất lượng đầu ra.
- **Đặc tính phục vụ:** đo thời gian phản hồi token đầu tiên, tốc độ sinh token, số yêu cầu đồng thời, độ ổn định và mức sử dụng GPU.
- **Khả năng tinh chỉnh:** kiểm tra chat template, tokenizer, hỗ trợ LoRA/QLoRA, yêu cầu dữ liệu và mức độ tương thích với công cụ huấn luyện.

Cách ước lượng VRAM

Ước lượng sơ bộ cho suy luận: VRAM cho trọng số ≈ số tham số × số bit mỗi tham số / 8. Sau đó cộng thêm KV cache, bộ đệm của runtime và phần dự phòng vận hành.

- **Khi suy luận:** KV cache tăng theo số phiên đồng thời, độ dài ngữ cảnh và kích thước batch; vì vậy phải benchmark trên tải gần với thực tế.
- **Khi fine-tune:** cần tính thêm activation, tham số adapter, gradient, trạng thái optimizer và chi phí do batch size, sequence length, gradient checkpointing tạo ra.
- **Nguyên tắc an toàn:** không cấu hình sát trần VRAM; duy trì khoảng trống để tránh lỗi hết bộ nhớ khi lưu lượng hoặc độ dài đầu vào tăng đột biến.

Các khoản chi phí phải tính

- GPU mua mới, khấu hao hoặc chi phí thuê theo giờ.
- Máy chủ CPU, RAM và SSD phục vụ model server, hàng đợi và các dịch vụ phụ trợ.
- Dung lượng tải model, lưu checkpoint, adapter và phiên bản dữ liệu.
- Điện năng, làm mát, không gian đặt máy hoặc phụ phí của nhà cung cấp hạ tầng.
- Băng thông mạng giữa backend, model server, kho lưu trữ và người dùng.
- Monitoring, tracing, cảnh báo, lưu log và thời gian lưu giữ dữ liệu quan sát.
- Backup dataset, checkpoint và adapter; chi phí kiểm tra khả năng khôi phục.
- Thời gian của kỹ sư cho triển khai, trực vận hành, xử lý sự cố và tối ưu hiệu năng.

- GPU hoặc phương án dự phòng khi máy chính lỗi, bảo trì hoặc quá tải.

Kết quả đầu ra: Bảng so sánh các model ứng viên, công thức ước lượng VRAM và tối thiểu ba kịch bản chi phí: pilot, tải mục tiêu và phương án dự phòng.

Giai đoạn 2: Xác định kiến thức cần học và nội dung có thể hoãn

Mục tiêu: Trang bị đủ năng lực để dựng, chạy, quan sát và xử lý lỗi của một model server GPU mà không sa vào các chủ đề chưa cần cho giai đoạn pilot.

Nội dung cần học trước

- **Linux:** quản lý file và quyền truy cập, tiến trình, service, biến môi trường, SSH, log, dung lượng đĩa và các lệnh chẩn đoán cơ bản.
- **Docker:** image, container, volume, network, Dockerfile, Compose, health check, log và chiến lược gắn phiên bản image.
- **Chạy model Hugging Face bằng Python:** tải model/tokenizer, áp dụng chat template, chọn dtype/device, sinh văn bản và đo thời gian, bộ nhớ.
- **Dựng Docker container có GPU:** NVIDIA driver và container runtime, ánh xạ GPU, kiểm tra bằng nvidia-smi, mount cache model và giới hạn tài nguyên.

Nội dung có thể bỏ qua ở giai đoạn đầu

- Tự huấn luyện foundation model từ đầu hoặc tự thiết kế kiến trúc transformer.
- Lập trình CUDA kernel và tối ưu mức phần cứng chuyên sâu.
- Huấn luyện phân tán nhiều node khi pilot vẫn chạy được trên một máy hoặc một cụm nhỏ.
- Kubernetes nâng cao trước khi lưu lượng và yêu cầu tự động mở rộng thực sự xuất hiện.
- Xây toàn bộ nền tảng MLOps tùy biến; ưu tiên công cụ tối giản nhưng có thể tái lập và quan sát được.

Kết quả đầu ra: Bốn bài thực hành có thể chạy lại từ đầu, kèm runbook cài đặt, kiểm tra GPU, khởi động container và xử lý các lỗi thường gặp.

Giai đoạn 3: Xây dựng kho dữ liệu Việt - Anh có kiểm duyệt

Mục tiêu: Tạo một nguồn dữ liệu có nguồn gốc rõ ràng, phù hợp mục đích, an toàn với trẻ em và đủ nhất quán để dùng cho SFT cũng như đánh giá.

Hướng dẫn gán nhãn

- Mục tiêu của từng nhãn, định nghĩa trường dữ liệu và tiêu chí chấp nhận hoặc loại bỏ mẫu.

- Quy tắc về độ chính xác ngôn ngữ, tính tự nhiên, mức độ phù hợp lứa tuổi, tính hữu ích và an toàn.
- Ví dụ đúng, ví dụ sai và các tình huống biên để giảm khác biệt giữa người gán nhãn.
- Quy trình xử lý bất đồng, kiểm tra chéo, đo mức nhất quán và cập nhật phiên bản hướng dẫn.
- Quy tắc không đưa thông tin nhận dạng cá nhân, nội dung nhạy cảm hoặc dữ liệu không đủ quyền sử dụng vào tập huấn luyện.

Yêu cầu riêng đối với dữ liệu trẻ em

- **Quyền và sự đồng ý:** chỉ sử dụng dữ liệu có căn cứ pháp lý, quyền khai thác và cơ chế đồng ý phù hợp; lưu bằng chứng về nguồn gốc dữ liệu.
- **Bảo vệ danh tính:** loại bỏ tên thật, địa chỉ, trường học, số điện thoại, giọng nói nhận dạng được và các dấu hiệu định danh khác khi không thật sự cần thiết.
- **Phù hợp lứa tuổi:** kiểm soát từ vựng, chủ đề, mức độ khó, cách phản hồi và cơ chế từ chối đối với nội dung nguy hiểm hoặc không phù hợp.
- **Tính đại diện:** cân bằng độ tuổi, vùng miền, giới tính, mức độ thành thạo và các kiểu phát âm; tránh củng cố định kiến.
- **Kiểm soát truy cập:** phân quyền kho dữ liệu, mã hóa khi lưu/truyền, ghi nhận truy cập, quy định thời gian lưu và quy trình xóa dữ liệu.

Quy trình dữ liệu đề xuất

1. Thu thập và kiểm tra quyền sử dụng dữ liệu.
2. Lọc trùng, chuẩn hóa định dạng và loại dữ liệu hỏng hoặc ngoài phạm vi.
3. Ẩn danh, quét thông tin cá nhân và đánh dấu nội dung nhạy cảm.
4. Gán nhãn, kiểm tra chéo và xử lý bất đồng theo hướng dẫn.
5. Chia train/validation/test theo nguyên tắc tránh rò rỉ nội dung hoặc người nói.
6. Phiên bản hóa dataset, dataset card, thống kê chất lượng và lịch sử thay đổi.

Kết quả đầu ra: Hướng dẫn gán nhãn, dataset card, tập dữ liệu đã phiên bản hóa, báo cáo kiểm duyệt và một bộ đánh giá chuẩn được khóa phiên bản.

Giai đoạn 4: Self-host Qwen và benchmark trước khi train

Mục tiêu: Tạo baseline có thể đo lường để xác định giới hạn của model gốc và chỉ fine-tune khi đã biết rõ vấn đề cần cải thiện.

Kiến trúc pilot

> **Ứng dụng/Backend Next.js -> Provider nội bộ -> Qwen Model Server -> GPU**
> **Monitoring, log và benchmark thu thập tín hiệu xuyên suốt luồng xử lý.**

Dựng model server

- Đóng gói inference server trong container, gắn phiên bản rõ ràng cho model, runtime và image.
- Cung cấp API hội thoại ổn định, ưu tiên giao diện tương thích OpenAI để đơn giản hóa tích hợp provider.
- Bổ sung health/readiness endpoint, xác thực nội bộ, giới hạn kích thước đầu vào, timeout và quản lý số phiên đồng thời.
- Tách model cache và log khỏi filesystem tạm của container; bảo đảm có quy trình khởi động lại và nạp model có kiểm soát.

Benchmark baseline

- **Hiệu năng:** thời gian phản hồi token đầu tiên, tổng thời gian trả lời, token/giây, throughput, mức dùng VRAM và tỷ lệ lỗi.
- **Chất lượng:** độ tự nhiên tiếng Việt - tiếng Anh, độ chính xác theo nhiệm vụ, khả năng tuân thủ hướng dẫn và xử lý hội thoại nhiều lượt.
- **An toàn:** phản hồi với dữ liệu cá nhân, nội dung trẻ em, yêu cầu nguy hiểm và tình huống model cần từ chối hoặc chuyển hướng.
- **Độ bền:** tải đồng thời, prompt dài, kết nối bị ngắt, model restart và hành vi khi GPU gần hết bộ nhớ.

Kết quả đầu ra: Model server gọi được qua API, bộ script benchmark có thể chạy lặp lại, báo cáo baseline và danh sách khoảng trống chất lượng cần giải quyết.

Giai đoạn 5: Fine-tune bằng SFT kết hợp QLoRA

Mục tiêu: Tinh chỉnh hành vi và phong cách hội thoại trên dữ liệu mục tiêu với chi phí VRAM thấp hơn, đồng thời giữ model gốc và adapter tách biệt để dễ thử nghiệm và hoàn tác.

Cơ chế fine-tune

- **SFT:** dạy model bắt chước câu trả lời chuẩn từ các cặp hội thoại đã kiểm duyệt; chất lượng dữ liệu và cách tạo nhãn loss ảnh hưởng trực tiếp đến kết quả.
- **QLoRA:** lượng tử hóa model nền trong quá trình huấn luyện và chỉ cập nhật các ma trận adapter LoRA, từ đó giảm đáng kể bộ nhớ cần dùng so với fine-tune toàn bộ tham số.

- **Nguyên tắc triển khai:** giữ nguyên checkpoint nền, lưu adapter theo phiên bản, ghi lại seed và cấu hình để có thể tái lập hoặc rollback.

Định dạng hội thoại

- Chuẩn hóa mỗi mẫu theo vai trò system, user và assistant; sử dụng đúng chat template/tokenizer của model Qwen được chọn.
- Quy định rõ điểm kết thúc lượt, xử lý hội thoại nhiều lượt, độ dài tối đa và cách loại các mẫu thiếu ngữ cảnh hoặc mâu thuẫn.
- Chỉ tính loss trên phần câu trả lời của assistant khi thiết kế huấn luyện yêu cầu như vậy; kiểm thử dữ liệu sau tokenize để tránh nhãn lệch hoặc mất token kết thúc.
- Giữ một bộ test cố định, không xuất hiện trong train, để so sánh model gốc và các adapter theo cùng một tiêu chí.

> **Ví dụ cấu trúc logic**
> system: Quy tắc vai trò và an toàn
> user: Câu hỏi hoặc lời nói của người học
> assistant: Phản hồi Việt - Anh đã được kiểm duyệt

Quy trình huấn luyện và đánh giá

1. Khóa phiên bản dataset, chat template, model nền và bộ đánh giá.
2. Chạy thử trên tập nhỏ để kiểm tra loss, tokenize, checkpoint và khả năng resume.
3. Thực hiện SFT + QLoRA với cấu hình được ghi nhận đầy đủ; theo dõi train/validation loss và tài nguyên GPU.
4. So sánh model gốc với adapter trên benchmark hiệu năng, chất lượng và an toàn.
5. Chỉ chọn adapter khi cải thiện mục tiêu chính mà không làm suy giảm đáng kể các tiêu chí quan trọng khác.

Kết quả đầu ra: Adapter đã phiên bản hóa, cấu hình và script huấn luyện tái lập được, log/checkpoint, model card và báo cáo so sánh trước - sau fine-tune.

Giai đoạn 6: Tích hợp backend Next.js và triển khai production

Mục tiêu: Tách biệt ứng dụng khỏi chi tiết của model server, bảo vệ cấu hình nhạy cảm và duy trì dịch vụ khi model chính chậm, lỗi hoặc quá tải.

Thiết kế provider

- Định nghĩa interface chung cho request, response, streaming, usage, timeout và mã lỗi; ứng dụng không gọi trực tiếp model server.

- Cài đặt provider cho Qwen self-host và provider dự phòng; chuẩn hóa lịch sử hội thoại, system prompt và giới hạn token tại một nơi.
- Tách logic retry, circuit breaker, rate limit và quan sát telemetry khỏi logic nghiệp vụ của API route.

Cấu hình server-side

- Lưu endpoint, API key, model ID, timeout, ngưỡng retry và cờ bật/tắt fallback trong biến môi trường chỉ dành cho server.
- Không đưa bí mật hoặc địa chỉ nội bộ vào client bundle; kiểm tra cấu hình ngay khi ứng dụng khởi động và thất bại sớm nếu thiếu giá trị bắt buộc.
- Phân tách cấu hình development, staging và production; quản lý secret bằng dịch vụ chuyên dụng và xoay vòng định kỳ.

Luồng fallback

1. Gửi yêu cầu tới Qwen self-host với timeout và mã truy vết duy nhất.
2. Nếu lỗi tạm thời, retry có backoff trong giới hạn cho phép; không retry các lỗi dữ liệu hoặc xác thực.
3. Khi vượt ngưỡng lỗi hoặc độ trễ, mở circuit breaker và chuyển sang provider dự phòng theo chính sách.
4. Nếu cả hai provider không khả dụng, trả thông báo an toàn, không làm mất trạng thái hội thoại và ghi sự kiện để điều tra.
5. Khi model chính phục hồi, thực hiện health probe và đưa lưu lượng trở lại từng bước.

Triển khai production

- **Tách dịch vụ:** backend Next.js và model server GPU triển khai độc lập để có thể mở rộng, cập nhật và rollback riêng.
- **Mạng và bảo mật:** TLS, mạng nội bộ, xác thực giữa dịch vụ, giới hạn truy cập, quét image và nguyên tắc đặc quyền tối thiểu.
- **Quan sát hệ thống:** metrics, log có cấu trúc, tracing, dashboard và cảnh báo cho độ trễ, tỷ lệ lỗi, hàng đợi, VRAM/GPU và chi phí.
- **Độ tin cậy:** health/readiness probe, graceful shutdown, rolling deployment, backup cấu hình/adapter và diễn tập khôi phục.
- **Kiểm soát phát hành:** staging, canary hoặc A/B test, bộ smoke test tự động và tiêu chí rollback rõ ràng.
- **Quản trị chi phí:** theo dõi chi phí theo môi trường, tải và số token; đặt cảnh báo ngân sách và lịch tắt tài nguyên không cần thiết.

Kết quả đầu ra: Provider hoàn chỉnh, cấu hình server-side an toàn, luồng fallback đã kiểm thử, dashboard/cảnh báo, runbook vận hành và quy trình triển khai - rollback.

Tiêu chí hoàn thành toàn bộ lộ trình

- Model và hạ tầng được chọn dựa trên benchmark và tổng chi phí sở hữu, không chỉ dựa trên số lượng tham số.
- Dữ liệu có nguồn gốc, quyền sử dụng, kiểm duyệt, hướng dẫn gán nhãn và phiên bản rõ ràng; dữ liệu trẻ em được bảo vệ theo chính sách riêng.
- Qwen self-host phục vụ ổn định qua API và có baseline hiệu năng, chất lượng, an toàn trước khi fine-tune.
- Adapter SFT + QLoRA cải thiện mục tiêu đã xác định và có thể tái lập, so sánh, triển khai hoặc rollback độc lập.
- Backend Next.js không phụ thuộc cứng vào một model; fallback, monitoring, backup và quy trình xử lý sự cố đã được kiểm thử.
- Hệ thống production có chủ sở hữu, SLO, cảnh báo, ngân sách và lịch rà soát định kỳ.

> **Nguyên tắc xuyên suốt.** Đo trước khi tối ưu, kiểm soát dữ liệu trước khi huấn luyện, và thiết kế khả năng phục hồi trước khi đưa hệ thống vào production.